Laboratório 08

IPv6 e Neighbor Discovery em Linux

Tecnologias de Comunicação para a Internet das Coisas | Aula 08

Objetivo e enquadramento

A Aula 08 apresenta o endereçamento IPv6, a autoconfiguração sem estado e o Neighbor Discovery Protocol, em contraste com o IPv4 e o NAT. Este laboratório torna essas ideias observáveis, construindo uma pequena rede com espaços de nomes de rede do Linux e capturando o tráfego real.

O trabalho decorre numa máquina virtual Ubuntu, criada e apagada com um único comando, e não requer acesso ao servidor da unidade curricular. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**.

Nota

Os espaços de nomes de rede são o mesmo mecanismo de isolamento que sustenta os containers. Cada um tem a sua própria pilha IPv6, com interfaces, endereços, rotas e tabela de vizinhos, pelo que se comporta como uma máquina distinta. A mesma sequência de comandos funciona num Raspberry Pi com Linux.

Resultados de aprendizagem

No final deste laboratório, o aluno deve ser capaz de:

- Identificar o endereço de ligação local de uma interface e relacioná-lo com o respetivo endereço MAC (EUI-64).
- Observar a troca de mensagens Neighbor Solicitation e Neighbor Advertisement e a tabela de vizinhos.
- Verificar o endereço multicast de nó solicitado associado a uma interface.

- Observar a autoconfiguração sem estado a partir de um Router Advertisement.
- Demonstrar o efeito do NAT em IPv4 e consultar a respetiva tabela de traduções.

Requisitos

- Computador com Windows, macOS ou Linux.
- Cerca de 1 GB de memória livre para a máquina virtual e 5 GB de disco.
- Ligação à Internet, necessária para obter a imagem e os pacotes.
- Duração estimada: entre 60 e 90 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

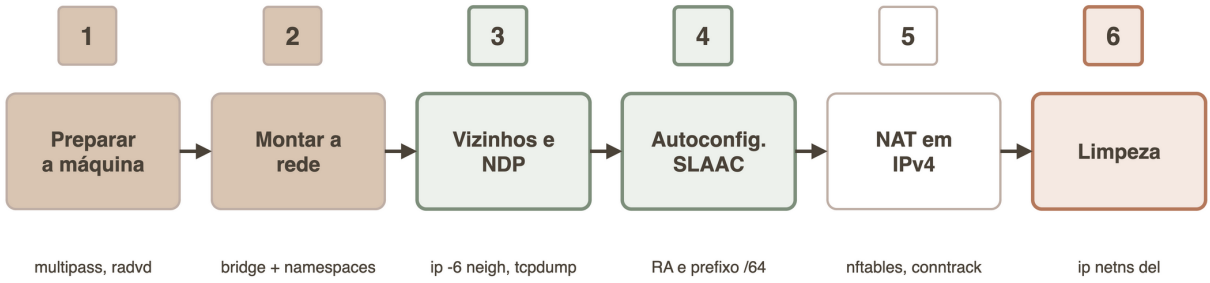
Erro que pode ocorrer e respetiva resolução.

Os comandos podem ser copiados e colados. Os resultados mostrados são exemplos: endereços MAC, endereços de ligação local e tempos diferem de computador para computador. Sempre

que um valor surja entre os sinais de menor e maior, como `<endereço>`, deve ser substituído, incluindo os sinais, pelo valor real apresentado no ecrã.

Percurso

A figura seguinte resume a sequência do guião. Os passos 3 e 4 constituem a observação central: o protocolo de descoberta de vizinhos e a autoconfiguração sem estado, em tráfego real.



Os passos 3 e 4 mostram, em tráfego real, o que a aula descreveu: NDP e autoconfiguração sem estado

Como ler a figura

Os seis passos leem-se da esquerda para a direita. Os dois primeiros, a bege, preparam o ambiente e constroem a rede. Os passos 3 e 4, a verde, são a parte central: observa-se como os nós se descobrem e como obtêm endereços sem servidor. O passo 5, a branco, mostra o NAT em IPv4 para contraste. O passo 6, avermelhado, encerra o trabalho. Abaixo de cada caixa, uma legenda indica a ferramenta principal do passo.

Passo 1 — Preparação da máquina

Objetivo. Criar uma máquina virtual Ubuntu e instalar as ferramentas de rede necessárias.

Enquadramento. O `multipass` cria máquinas virtuais Ubuntu com um único comando e funciona em Windows, macOS e Linux. As ferramentas `radvd`, `tcpdump` e `conntrack` serão usadas nos passos seguintes.

Fundamento teórico

Uma máquina virtual isola o laboratório do sistema do aluno: os espaços de nomes, as pontes e as regras de firewall criadas existem apenas dentro dela e desaparecem quando é apagada. O `radvd` é um programa que anuncia prefixos IPv6 através de mensagens Router Advertisement, e desempenha neste laboratório o papel do router que o diapositivo da autoconfiguração descreve. O `tcpdump` apresenta os pacotes que atravessam uma interface, e o `conntrack` lista a tabela de traduções do NAT.

A analisar neste passo: apenas a preparação. Nada do que se instala interfere com a rede do computador.

1.1 Criação da máquina virtual

Em macOS, instalar com `brew install --cask multipass`. Em Windows, utilizar o instalador de https://multipass.run. Em Linux, `sudo snap install multipass`. Depois:

Comando a executar

```
multipass launch 22.04 --name tciot-ipv6 --cpus 2 --memory 1G --disk 5G
multipass shell tciot-ipv6
```

Resultado esperado (os valores podem variar)

```
Launched: tciot-ipv6
ubuntu@tciot-ipv6:~$
```

A partir deste momento, todos os comandos são executados dentro da máquina virtual.

1.2 Instalação das ferramentas

Comando a executar

```
sudo apt update
sudo apt install -y radvd tcpdump conntrack nftables curl
```

Erro possível e resolução

launch failed ou execução parada: o computador não dispõe de memória livre suficiente. Fechar outras aplicações e repetir. Se a máquina já existir, executar `multipass delete tciot-ipv6`, `multipass purge` e repetir.

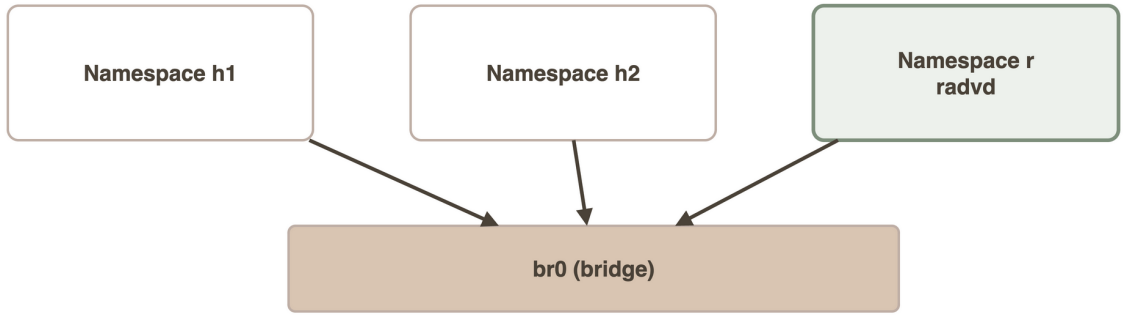
Verificação antes de avançar

O comando `tcpdump --version` apresenta a versão instalada.

Passo 2 — Montagem da rede

Objetivo. Criar três máquinas lógicas, `h1`, `h2` e `r`, ligadas ao mesmo segmento de camada 2.

Enquadramento. Cada espaço de nomes de rede comporta-se como uma máquina. Uma ponte Linux, a `br0`, desempenha o papel de um comutador que as liga.



Todos ligados ao mesmo segmento de camada 2; o router anuncia o prefixo 2001:db8:1::/64

Como ler a figura

Os três espaços de nomes estão em cima e a ponte, a bege, por baixo. Cada seta representa um cabo virtual entre a interface `eth0` do espaço de nomes e a ponte. O `r` está a verde porque será o router, e o `h1` e o `h2` são máquinas terminais. A caixa inferior indica o prefixo que o router anunciará no passo 4.

Fundamento teórico

Um par *veth* é um cabo virtual: o que entra numa extremidade sai na outra. Uma das extremidades fica no espaço de nomes, com o nome `eth0`, e a outra é ligada à ponte. Quando uma interface é ativada, o núcleo atribui-lhe de imediato um endereço de ligação local, com prefixo `fe80::/10`, e executa a deteção de endereços duplicados. Por isso, ao fim de um ou dois segundos, cada máquina já tem endereço IPv6 sem qualquer configuração.

A analisar neste passo: o que existe antes de qualquer configuração manual. Nenhum endereço foi atribuído, e as interfaces ficam, ainda assim, com um endereço IPv6.

2.1 Criação da ponte e dos espaços de nomes

Comando a executar

```
sudo ip link add br0 type bridge
sudo ip link set br0 type bridge mcast_snooping 0
sudo ip link set br0 up
for n in h1 h2 r; do
  sudo ip netns add $n
  sudo ip link add v-$n type veth peer name eth0 netns $n
  sudo ip link set v-$n master br0
  sudo ip link set v-$n up
  sudo ip netns exec $n ip link set lo up
  sudo ip netns exec $n ip link set eth0 up
done
```

2.2 Verificação

Comando a executar

```
sudo ip netns list
sudo ip netns exec h1 ip -6 addr show dev eth0
```

Resultado esperado (os valores podem variar)

```
r
h2
h1
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    inet6 fe80::c4a:1eff:fe3c:4d5e/64 scope link
       valid_lft forever preferred_lft forever
```

Erro possível e resolução

Cannot find device ou `File exists`: um comando anterior falhou ou foi repetido. Remover o que ficou a meio com `sudo ip netns del h1` (e `h2`, `r`) e `sudo ip link del br0`, e repetir o passo.

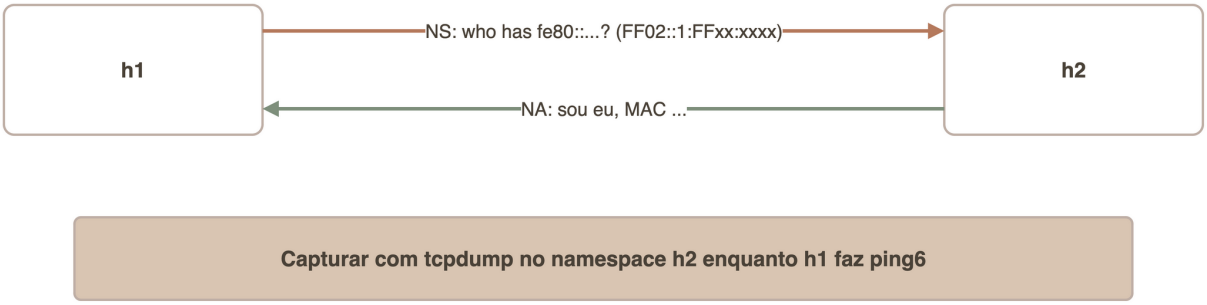
Verificação antes de avançar

Existem três espaços de nomes e cada `eth0` tem um endereço que começa por `fe80::`.

Passo 3 — Vizinhos e Neighbor Discovery

Objetivo. Observar o endereço de ligação local, a tabela de vizinhos e a troca de mensagens Neighbor Solicitation e Neighbor Advertisement.

Enquadramento. Em IPv6 o ARP não existe. O mesmo trabalho é feito pelo Neighbor Discovery Protocol, com mensagens ICMPv6 enviadas para endereços multicast de nó solicitado.



Como ler a figura

`h1` quer enviar um pacote a `h2`, mas ainda não conhece o seu endereço físico. A seta superior é a Neighbor Solicitation, enviada para o endereço multicast de nó solicitado de `h2`. A seta inferior é a Neighbor Advertisement, em unicast, que traz o endereço físico. A caixa inferior indica onde a captura é feita: em `h2`, enquanto `h1` envia o ping.

Fundamento teórico

Na autoconfiguração clássica, o identificador de interface deriva do endereço MAC: insere-se FFFE a meio e inverte-se o bit universal/local. É por isso que o endereço de ligação local de uma interface com MAC `aa:bb:cc:dd:ee:ff` contém `a8bb:ccff:fedd:eeff`. Os núcleos mais recentes podem usar identificadores opacos, e nesse caso a relação não é visível.

Para resolver o endereço físico de um vizinho, o nó envia uma Neighbor Solicitation para o endereço multicast de nó solicitado, formado por `ff02::1:ff` e pelos 24 bits finais do endereço procurado. Só os nós cujo endereço acaba nos mesmos 24 bits processam a mensagem. O resultado é guardado na tabela de vizinhos, onde cada entrada tem um estado, como `REACHABLE` ou `STALE`.

A analisar neste passo: a relação entre MAC e endereço, e o facto de a solicitação não ser difundida a toda a rede.

3.1 Endereços e relação com o MAC

Comando a executar

```
sudo ip netns exec h1 ip link show eth0 | grep ether
sudo ip netns exec h1 ip -6 addr show dev eth0 scope link
sudo ip netns exec h2 ip -6 addr show dev eth0 scope link
```

Analisar

Compare o endereço MAC de `h1` com os 64 bits finais do seu endereço de ligação local. Que passos permitem passar de um para o outro? Que alteração sofreu o primeiro octeto?

3.2 Ping a todos os nós

Comando a executar

```
sudo ip netns exec h1 ping -6 -c 2 -I eth0 ff02::1
```

Resultado esperado (os valores podem variar)

```
64 bytes from fe80::c4a:1eff:fe3c:4d5e%eth0: icmp_seq=1 ttl=64 time=0.07 ms
64 bytes from fe80::7c2d:9ff:fe11:2233%eth0: icmp_seq=1 ttl=64 time=0.31 ms (DUP!)
64 bytes from fe80::a4b8:5cff:fe90:1a77%eth0: icmp_seq=1 ttl=64 time=0.33 ms (DUP!)
```

O endereço `ff02::1` designa todos os nós da ligação. Responderam o próprio `h1`, o `h2` e o `r`. Os avisos `DUP!` são esperados: uma só mensagem teve várias respostas.

3.3 Captura da descoberta

Obter o endereço de ligação local de `h2` e limpar a tabela de vizinhos de `h1`:

Comando a executar

```
sudo ip netns exec h2 ip -6 addr show dev eth0 scope link | grep inet6
sudo ip netns exec h1 ip -6 neigh flush dev eth0
```

Num segundo terminal da máquina virtual, iniciar a captura em `h2`:

Comando a executar

```
sudo ip netns exec h2 tcpdump -ni eth0 -c 4 icmp6
```

No primeiro terminal, enviar um pedido a `h2`, substituindo o endereço:

Comando a executar

```
sudo ip netns exec h1 ping -6 -c 1 -I eth0 <endereco-de-h2>
```

Resultado esperado (os valores podem variar)

```
IP6 fe80::c4a:1eff:fe3c:4d5e > ff02::1:ff11:2233: ICMP6, neighbor solicitation, who has
    fe80::7c2d:9ff:fe11:2233
IP6 fe80::7c2d:9ff:fe11:2233 > fe80::c4a:1eff:fe3c:4d5e: ICMP6, neighbor advertisement, tgt
     is fe80::7c2d:9ff:fe11:2233
IP6 fe80::c4a:1eff:fe3c:4d5e > fe80::7c2d:9ff:fe11:2233: ICMP6, echo request, id 17, seq 1
IP6 fe80::7c2d:9ff:fe11:2233 > fe80::c4a:1eff:fe3c:4d5e: ICMP6, echo reply, id 17, seq 1
```

Analisar

Qual é o endereço de destino da primeira mensagem? Compare os seus 24 bits finais com os do endereço de `h2`. Que mensagem desempenha, em IPv6, o papel do pedido ARP?

3.4 Tabela de vizinhos e grupos multicast

Comando a executar

```
sudo ip netns exec h1 ip -6 neigh
sudo ip netns exec h2 ip maddr show dev eth0
```

Resultado esperado (os valores podem variar)

```
fe80::7c2d:9ff:fe11:2233 dev eth0 lladdr 7e:2d:09:11:22:33 REACHABLE
    inet6 ff02::1:ff11:2233
    inet6 ff02::1
    inet6 ff01::1
```

Erro possível e resolução

A captura não mostra a Neighbor Solicitation: a tabela de vizinhos de `h1` já tinha a entrada. Repetir o comando `neigh flush` imediatamente antes do ping.
Network is unreachable: faltou indicar a interface com `-I eth0`. Os endereços de ligação local só são válidos na interface respetiva.

Verificação antes de avançar

Foi visto o par Neighbor Solicitation e Neighbor Advertisement, e o grupo `ff02::1:ff...` aparece na lista de grupos de `h2`.

Passo 4 — Autoconfiguração sem estado

Objetivo. Fazer com que o router anuncie um prefixo e observar os hosts a construírem endereços globais sem servidor.

Enquadramento. É a característica do IPv6 mais valiosa para IoT: um dispositivo liga-se e obtém endereço, sem configuração manual e sem DHCP.

Fundamento teórico

O router envia periodicamente, e em resposta a uma Router Solicitation, mensagens Router Advertisement com o prefixo da rede. Se a opção *autonomous* estiver ativa, cada host concatena o prefixo de 64 bits com o seu identificador de interface e obtém um endereço global, após executar a deteção de duplicados. O router não guarda registo das atribuições, e por isso se diz que a autoconfiguração é sem estado. A mesma mensagem anuncia ainda o router como gateway por omissão, e o host acrescenta uma rota para ele.

A analisar neste passo: quem sabe o quê. O router conhece apenas o prefixo; cada host calcula o seu próprio endereço.

4.1 Configuração do router

Criar o ficheiro `radvd.conf` na pasta pessoal da máquina virtual, por exemplo com o `nano`:

Conteúdo do ficheiro `radvd.conf`

```
interface eth0 {
    AdvSendAdvert on;
    MinRtrAdvInterval 3;
    MaxRtrAdvInterval 10;
    prefix 2001:db8:1::/64 {
        AdvOnLink on;
        AdvAutonomous on;
    };
};
```

O prefixo `2001:db8::/32` está reservado para documentação, pelo que é seguro em testes.

4.2 Execução e captura

Comando a executar

```
sudo ip netns exec r sysctl -w net.ipv6.conf.all.forwarding=1
sudo ip netns exec r radvd -n -m stderr -C ~/radvd.conf
```

O terminal fica ocupado. Num segundo terminal, verificar os endereços e a rota de `h1`:

Comando a executar

```
sudo ip netns exec h1 ip -6 addr show dev eth0
sudo ip netns exec h1 ip -6 route
```

Resultado esperado (os valores podem variar)

```
inet6 2001:db8:1:0:c4a:1eff:fe3c:4d5e/64 scope global dynamic mngtmpaddr
inet6 fe80::c4a:1eff:fe3c:4d5e/64 scope link
default via fe80::a4b8:5cff:fe90:1a77 dev eth0 proto ra metric 1024 expires 1790sec
2001:db8:1::/64 dev eth0 proto kernel metric 256 expires 86390sec
```

Analisar

O endereço global usa o prefixo anunciado e os mesmos 64 bits finais do endereço de ligação local. Onde estava a informação do prefixo antes do anúncio? Quem escolheu os 64 bits finais?

4.3 Comunicação com endereço global

Comando a executar

```
sudo ip netns exec h2 ip -6 addr show dev eth0 | grep global
sudo ip netns exec h1 ping -6 -c 2 <endereco-global-de-h2>
```

Erro possível e resolução

O host não obtém endereço global: confirmar que o `radvd` está a correr e que `forwarding` está a 1 em `r`. Em `h1`, `sysctl net.ipv6.conf.eth0.accept_ra` deve devolver 1.
radvd: config file permissions: o ficheiro não pode ser gravável por outros utilizadores.

Executar `chmod 644 ~/radvd.conf`.

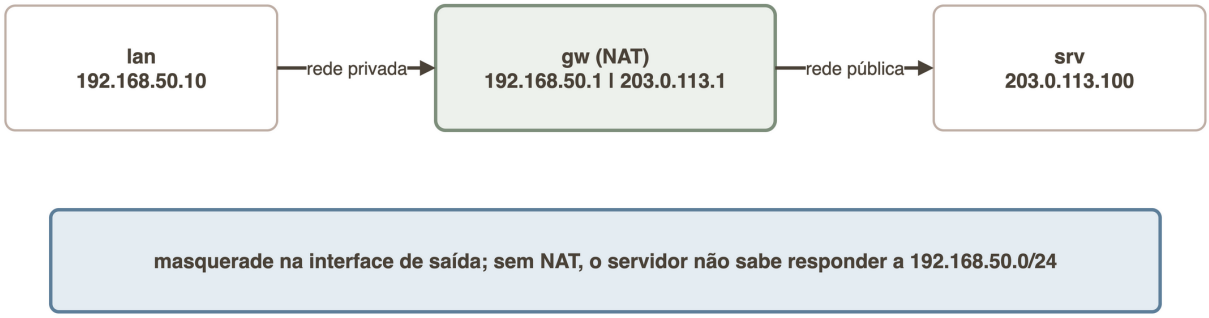
Verificação antes de avançar

`h1` e `h2` têm endereço global com o prefixo `2001:db8:1::/64` e comunicam por ele.

Passo 5 — NAT em IPv4, para contraste

Objetivo. Demonstrar que uma máquina com endereço privado só comunica com o exterior através de tradução de endereços, e consultar a tabela de NAT.

Enquadramento. A aula apresentou o NAT como remendo para o esgotamento de endereços IPv4. Aqui o seu efeito é observado de ambos os lados.



Como ler a figura

À esquerda está a máquina da rede privada, `lan`. Ao centro está o `gw`, com duas interfaces: uma na rede privada e outra na rede pública. À direita está o servidor `srv`, que só conhece a rede pública. A caixa inferior resume o que se vai demonstrar: sem tradução, o servidor recebe pedidos de um endereço para o qual não tem rota e não consegue responder.

Fundamento teórico

Os endereços da gama 192.168.0.0/16 são privados e não são encaminhados na Internet. Quando um pacote com esse endereço de origem chega a um servidor público, este não sabe como responder. O *masquerade* substitui, em cada pacote que sai, o endereço de origem pelo da interface de saída do router e regista a correspondência no *connection tracking*. Na resposta, o registo permite a operação inversa.

A analisar neste passo: o endereço que o servidor vê. Não é o da máquina que fez o pedido.

5.1 Topologia

Em primeiro lugar, remover a rede do passo 2 e parar o `radvd` (`Ctrl+C` no terminal onde corre):

Comando a executar

```
for n in h1 h2 r; do sudo ip netns del $n; done
sudo ip link del br0
```

Comando a executar

```
for n in lan gw srv; do sudo ip netns add $n; done
sudo ip link add l-lan type veth peer name l-gw
sudo ip link add w-gw type veth peer name w-srv
sudo ip link set l-lan netns lan
sudo ip link set l-gw netns gw
sudo ip link set w-gw netns gw
sudo ip link set w-srv netns srv
sudo ip netns exec lan ip addr add 192.168.50.10/24 dev l-lan
sudo ip netns exec gw ip addr add 192.168.50.1/24 dev l-gw
sudo ip netns exec gw ip addr add 203.0.113.1/24 dev w-gw
sudo ip netns exec srv ip addr add 203.0.113.100/24 dev w-srv
for p in "lan l-lan" "gw l-gw" "gw w-gw" "srv w-srv"; do
  set -- $p; sudo ip netns exec $1 ip link set $2 up
done
sudo ip netns exec lan ip route add default via 192.168.50.1
sudo ip netns exec gw sysctl -w net.ipv4.ip_forward=1
```

5.2 Pedido sem NAT

Iniciar um servidor HTTP em `srv`, noutro terminal:

Comando a executar

```
sudo ip netns exec srv python3 -m http.server 8000
```

Comando a executar

```
sudo ip netns exec lan curl -s -m 5 -o /dev/null -w "%{http_code}\n" http
    ://203.0.113.100:8000/
```

Resultado esperado (os valores podem variar)

```
000
```

O pedido chega ao servidor, mas a resposta não regressa, porque o `srv` não tem rota de regresso para a rede privada.

5.3 Pedido com NAT

Comando a executar

```
sudo ip netns exec gw nft add table ip nat
sudo ip netns exec gw nft add chain ip nat post '{ type nat hook postrouting
    priority 100 ; }'
sudo ip netns exec gw nft add rule ip nat post oifname "w-gw" masquerade
sudo ip netns exec lan curl -s -m 5 -o /dev/null -w "%{http_code}\n" http
    ://203.0.113.100:8000/
```

Resultado esperado (os valores podem variar)

```
200
```

No terminal do servidor, o registo de acessos mostra o endereço de origem:

Resultado esperado (os valores podem variar)

```
203.0.113.1 - - [14/Oct/2026 10:21:07] "GET / HTTP/1.1" 200 -
```

5.4 Tabela de traduções

Comando a executar

```
sudo ip netns exec gw conntrack -L
```

Resultado esperado (os valores podem variar)

```
tcp  6 118 TIME_WAIT src=192.168.50.10 dst=203.0.113.100 sport=45822 dport=8000
     src=203.0.113.100 dst=203.0.113.1 sport=8000 dport=45822 [ASSURED]
```

Analisar

O servidor registou 203.0.113.1 como origem do pedido, e não 192.168.50.10. Onde ficou guardada a informação que permite devolver a resposta à máquina certa? Que problema surgiria se o `srv` quisesse iniciar uma ligação para o `lan`?

Erro possível e resolução

curl devolve 000 mesmo com NAT: confirmar que `ip_forward` está a 1 em `gw` e que o servidor HTTP continua ativo. **conntrack: command not found**: instalar com `sudo apt install -y conntrack`.

Verificação antes de avançar

O pedido falhou sem NAT, funcionou com NAT, e a tabela `conntrack` mostra as duas versões da ligação.

Passo 6 — Limpeza

Objetivo. Parar os serviços e apagar a rede e a máquina virtual.

6.1 Dentro da máquina virtual

Comando a executar

```
sudo pkill -f "http.server 8000"
sudo pkill radvd
for n in lan gw srv h1 h2 r; do sudo ip netns del $n 2>/dev/null; done
sudo ip link del br0 2>/dev/null
exit
```

6.2 No computador do aluno

Comando a executar

```
multipass delete tciot-ipv6
multipass purge
multipass list
```

Resultado esperado (os valores podem variar)

```
No instances found.
```

Nota

Para repetir o laboratório, basta repetir os passos 1 a 5. Os ficheiros `radvd.conf` e os comandos ficam neste guião.

Resolução de problemas: consulta rápida

Sintoma	Resolução
`multipass: command not found`	Fechar e abrir o terminal. Se persistir, repetir a instalação.
`File exists` ou `Cannot find device`	Remover espaços de nomes e ponte com `ip netns del` e `ip link del br0`, e repetir.
Endereço em estado `tentative`	Aguardar um a dois segundos pela deteção de duplicados.
Ping sem resposta com `fe80::`	Indicar a interface: `-I eth0` ou `%eth0`.
Sem Neighbor Solicitation na captura	Executar `ip -6 neigh flush dev eth0` antes do ping.
Sem endereço global	Verificar o `radvd` e `forwarding=1` em `r`.
`radvd: config file permissions`	`chmod 644 ~/radvd.conf`.
`curl` devolve `000` com NAT	Verificar `ip_forward`, a regra `masquerade` e o servidor HTTP.

Comandos utilizados

Comando	Finalidade
`ip netns add / exec / del`	Criar, usar e apagar um espaço de nomes
`ip link add ... type veth`	Criar um cabo virtual
`ip -6 addr show`	Listar endereços IPv6
`ip -6 neigh`	Ver a tabela de vizinhos
`ip maddr show`	Listar grupos multicast
`ping -6 -I eth0 ff02::1`	Alcançar todos os nós da ligação
`tcpdump -ni eth0 icmp6`	Capturar mensagens ICMPv6
`radvd -C radvd.conf`	Anunciar um prefixo
`nft add rule ... masquerade`	Ativar NAT de saída
`conntrack -L`	Listar a tabela de traduções

Questões de consolidação

As questões seguintes destinam-se a discussão e não têm de ser entregues.

1. Como se relaciona o endereço de ligação local de uma interface com o seu endereço MAC?
2. Que endereço de destino usou a Neighbor Solicitation e porque não foi difundida a toda a rede?
3. Quem decidiu o prefixo e quem decidiu os 64 bits finais do endereço global?
4. Que informação perderia o servidor sem NAT, e que informação guarda o `gw` para devolver a resposta?
5. Que vantagem tem a autoconfiguração sem estado numa instalação com centenas de sensores?

Síntese

O laboratório permitiu observar, em tráfego real, as ideias centrais da Aula 08:

- Cada interface IPv6 obtém um **endereço de ligação local** sem configuração, derivado do MAC.
- O **Neighbor Discovery** substitui o ARP, com solicitações enviadas para o endereço multicast de nó solicitado.
- A **autoconfiguração sem estado** permite obter endereço global a partir do prefixo anunciado, sem servidor.
- O **NAT** permite a máquinas privadas comunicar com o exterior, mas guarda estado e esconde a origem real.